# TALLER 2
# TALLER INTEROPERABILIDAD ADAPTADOR DE INTERFAZ

**Nombre**


- Alixon Julieth López Alvarado
- Robinson Felipe Molina Granados


***Objetivo**:* Implementar una estrategia de interoperabilidad mediante la táctica de Adaptador, garantizando el desacoplamiento estructural, la estabilidad del contrato interno y la resistencia ante cambios externos.

**Link del repositorio*:*** https://github.com/RobinsonMolina/Order-system-interoperability.git

**Contexto del sistema**

Una empresa tiene un sistema principal de órdenes que depende de un proveedor externo de pagos. El sistema interno espera el siguiente contrato estable: procesarPago(clienteId, monto) → { estado, codigoAutorizacion }

Sin embargo, el proveedor externo expone la siguiente interfaz incompatible: executeTransaction(user, amount, currency) → { resultCode, authId, timestamp }

El proveedor puede cambiar los nombres de los campos, agregar nuevos parámetros o modificar la estructura de la respuesta sin previo aviso. El sistema principal no puede modificarse ante esos cambios

**DOCUMENTO TÉCNICO**

*Requisito principal*: El sistema principal debe crear órdenes y procesar pagos sin depender directamente de la estructura real del proveedor externo.

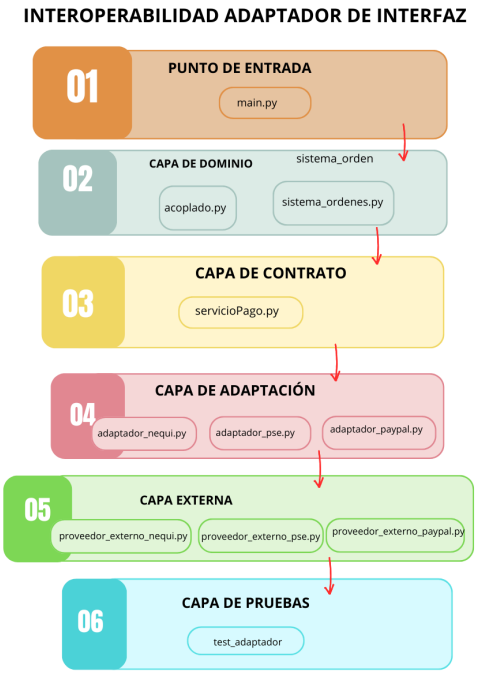


*Figura 1. Diagrama simple de arquitectura*

## PROCEDIMIENTO

**Paso 1 –** Implementación ingenua (acoplada)

Implemente el sistema principal conectándolo directamente al proveedor externo. Demuestre que funciona y documente el nivel de acoplamiento.

La implementación de conectar el sistema principal con el proveedor PSE de manera directa se lleva a cabo de manera sencilla, sin embargo, en esta, el nivel de acoplamiento es demasiado alto. Se implementaron tres componentes, en primer lugar el proveedor PSE (proveedor_externo_pse.py) que simula el servicio externo de pagos, luego el sistema de órdenes acoplado (acoplado.py) que crea las órdenes llamando directamente al proveedor y un breve menú por consola (main.py) que permite el flujo de forma interactiva.

## SISTEMA DE ÓRDENES ACOPLADO

El sistema de órdenes impronta el proveedor PSE de manera directa, llamando o utilizando executeTransaction con los parámetros del proveedor, posteriormente lee el campo resultCode para verificar el estado y retorna el ordenId, el estado y el código de autorización.

```python
from proveedor.proveedor_externo_pse import PSE

class SistemaOrdenesAcoplado:

    CODIGO_APROBADO = "00"

    def __init__(self):
        self._proveedor = PSE()

    def crearOrden(self, cliente_id: str, monto: float) -> dict:
        respuesta = self._proveedor.executeTransaction(
            user=cliente_id,
            amount=monto,
            currency="COP"
        )

        aprobado = respuesta["resultCode"] == self.CODIGO_APROBADO

        return {
            "ordenId":            f"ORD-{cliente_id}-001",
            "estado":             "APROBADO" if aprobado else "RECHAZADO",
            "codigoAutorizacion": respuesta["authId"]
        }
```

*Figura 2. Sistema de órdenes acoplado.*

## MÉTODO QUE PERMITE LA INVOCACIÓN DE CREAR ORDEN

Este método permite la invocación al método crearOrden, se utiliza para que el código sea más ordenado en pasos posteriores y por buenas prácticas.

```python
#crear una orden usando el sistema acoplado
def crear_orden_acoplada(cliente_id: str, monto: float) -> dict:
    return SistemaOrdenesAcoplado().crearOrden(cliente_id, monto)
```

*Figura 3.Método directo para crear orden.*

## PROVEEDOR EXTERNO DEFINIDO COMO PSE

El proveedor suma dos escenarios distintos, el primero hace referencia a un caso con monto de menos de 10.000.000 (diez millones), en donde la orden y estado se da como aprobado, en caso contrario, es decir, en caso de que la orden esté por encima de 10.000.000 su estado será rechazado.

```python
class PSE:

    #respuesta del proveedor
    CODIGO_APROBADO  = "00"
    CODIGO_RECHAZADO = "99"
    LIMITE_MONTO = 10_000_000

    def executeTransaction(self, user: str,amount: float,currency: str) -> dict:
        print(
            f"  [PSE] executeTransaction("
            f"user='{user}', amount={amount:,.0f}, currency='{currency}')"
        )

        timestamp = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

        if amount <= self.LIMITE_MONTO:
            return {
                "resultCode": self.CODIGO_APROBADO,
                "authId":     "PSE-AUTH-1289",
                "timestamp":  timestamp
            }

        return {
            "resultCode": self.CODIGO_RECHAZADO,
            "authId":     None,
            "timestamp":  timestamp
        }
```

*Figura 4. Proveedor externo (PSE).*

El método devuelve 3 campos de respuesta, resultCode que es el código resultado(00 para aprobado y 51 para rechazado), authId que es el ID de la autorización y timestamp que indica la fecha y hora.

## RESULTADO DE FUNCIONAMIENTO

En las siguientes imágenes se visualiza que el sistema funciona de manera correcta en ambos casos, cuando el monto está dentro del límite y cuando no lo está devolviendo la respuesta esperada.

```
TALLER II — INTEROPERABILIDAD ADAPTADOR DE INTERFAZ

1. Implementacion Ingenuea Acoplamiento — PSE

Selecciona una opción: 1

OPCIÓN 1 — Acoplamiento directo con PSE

Cliente ID (Enter = 'cliente-001'): cliente-002
Monto COP  (Enter = 5000000)     : 100000
Llamada al proveedor:
[PSE] executeTransaction(user='cliente-002', amount=100,000, currency='COP')

 Pago APROBADO
Orden ID            : ORD-cliente-002-001
Código autorización : PSE-AUTH-1289
```

```
TALLER II — INTEROPERABILIDAD ADAPTADOR DE INTERFAZ

1. Implementacion Ingenuea Acoplamiento — PSE

Selecciona una opción: 1

OPCIÓN 1 — Acoplamiento directo con PSE

Cliente ID (Enter = 'cliente-001'): cliente-003
Monto COP  (Enter = 5000000)     : 25000000
Llamada al proveedor:
[PSE] executeTransaction(user='cliente-003', amount=25,000,000, currency='COP')

 Pago RECHAZADO
Orden ID            : ORD-cliente-003-001
Código autorización : —
```

*Figura 5. Resultado de ejecución, aprobado y rechazado.*

En resumen esta implementación funciona porque ambos utilizan la misma estructura de datos, el sistema conoce como responde el proveedor y al estar conectados de manera directa tienen un acoplamiento fuerte. Sin embargo, si por alguna razón se modifica el proveedor el sistema ya no funcionará de igual manera sino que dará como resultado un error.

**Paso 2 –** Análisis de impacto del cambio

Simule un cambio en el proveedor (renombrar campos o modificar la estructura). Identifique qué partes del sistema principal deben modificarse.

**NUEVO PROVEEDOR PAYPAL**

Se realizaron un total de 4 cambios en el proveedor Paypal, de manera general el código es igual al de proveedor PSE, pero con las siguientes modificaciones:

1. Se agregaron dos parámetro obligatorios correspondiente a la descripción y producto de la orden.
2. Se eliminó el campo de timestamp de la respuesta del proveedor.
3. El campo resultCode se renombró por estado.
4. El campo authID se renombró con autorización.

Order-system-interoperability > proveedor > proveedor_externo_paypal.py > Paypal > executeTransaction

```python

oveedor externo con cambios para el paso 2 - PayPal
s Paypal:

#respuesta del proveedor
CODIGO_APROBADO  = "APROBADO"
CODIGO_RECHAZADO = "RECHAZADO"
LIMITE_MONTO = 10_000_000

def executeTransaction(self, user: str,amount: float,currency: str, descripcion: str, producto: int )
    print(
        f"  [PayPal] executeTransaction("
        f"user='{user}', amount={amount:,.0f}, currency='{currency}, descripcion='{descripcion}', pro
    )

    if amount <= self.LIMITE_MONTO:
        return {
            "estado": self.CODIGO_APROBADO,
            "autorizacion": "PAYPAL-AUTH-4567"
        }

    return {
        "estado": self.CODIGO_RECHAZADO,
        "autorizacion":     None
    }
}
```

*Figura 6. Cambios en el proveedor.*

**MÉTODO EN SISTEMA PRINCIPAL**

Este método se implementó en el sistema principal, de esta manera se indica el cambio de proveedor.

```python
#crear una orden con cambios en el proveedor
def orden_acoplada_paypal(cliente_id: str, monto: float) -> dict:
    return SistemaOrdenesAcoplado(Paypal()).crearOrden(cliente_id, monto)
```

*Figura 7. Innovación de crear orden con proveedor Paypal.*

**RESULTADO FUNCIONAMIENTO**

El resultado de ejecución muestra un TypeError indicando que hacen faltan argumentos obligatorios, además muestra la causa y menciona que para que la operación funciones se deben realizar los

respectivos cambios en el sistema principal referencias a agregar la descripción, el producto, cambiar resultCode por estado y authId por autorización.

```
PASO 2 — Análisis de impacto del cambio (PayPal)

Cliente ID (Enter = 'cliente-001'): cliente-004
Monto COP  (Enter = 5000000)     : 560000

Llamada al proveedor:
 TypeError → Paypal.executeTransaction() missing 2 required positional arguments: 'descripcion' and 'producto'

Causa: PayPal exige 'descripcion' y 'producto' pero el sistema no lo envía.
Para corregirlo habría que modificar el sistema principal.
```

*Figura 8. Resultado de ejecución.*

**Paso 3 –** Definición de contrato interno estable

Defina una interfaz interna (ej. ServicioPago) que represente el contrato establece que utilizará el sistema principal. El sistema principal solo puede depender de esta interfaz.

**IMPLEMENTACIÓN INTERFAZ**

Se define la clase abstracta o interfaz que define el contrato de procesarPago, lo que quiere decir que cualquier sistema de pago debe implementar esta interfaz.

```python
from abc import ABC, abstractmethod

class ServicioPago(ABC):

    @abstractmethod
    def procesarPago(self, clienteId: str, monto: float) -> dict:
        pass
```

*Figura 9. Implementación de interfaz Servicio Pago - contrato interno.*

**SISTEMA DE ÓRDENES ACTUALIZADO CON USO DE INTERFAZ**

Esta clase recibe un objetivo que cumple el contrato de la interfaz y lo utiliza para procesar el pago cuando se crea una orden, todo esto independiente del proveedor de pago, es decir, en esta no se tiene en cuenta si es PSE o PayPal, pero aun así va a funcionar.

```python
from interfaz.servicioPago import ServicioPago

class SistemaOrdenes:

    def __init__(self, servicio_pago: ServicioPago):
        self._servicio = servicio_pago

    def crearOrden(self, cliente_id: str, monto: float) -> dict:

        resultado = self._servicio.procesarPago(cliente_id, monto)

        return {
            "ordenId":            f"ORD-{cliente_id}-001",
            "estado":             resultado["estado"],
            "codigoAutorizacion": resultado["codigoAutorizacion"]
        }
```

*Figura 10. Nuevo sistema principal.*

**Paso 4 –** Implementación del Adaptador

Se implementaron 3 adaptadores para cada uno de los proveedores (PSE, Paypal, Nequi). estos actúan como un traductor entre el sistema de órdenes y los proveedores de la siguiente manera::

- Traduce los parámetros internos al formato externo. El sistema interno siempre envía dos parametros (clienteId y monto). Sin embargo, cada uno de los proveedores exige nombres o datos distintos. El adaptador hace la conversión antes de comunicarse con el proveedor.

  Por ejemplo: Mientras el sistema interno usa 'monto', tanto PSE como Paypal usan 'amount' y Nequi usa 'value'. Además el adaptador inyecta automáticamente los parámetros obligatorios de 'descripcion' y 'producto' en Paypal, que el sistema de órdenes no reconoce pero es necesario en el proveedor

- Convierte la respuesta externa al formato interno: Las respuestas de los proveedores son distintas, el adaptador las transforma a una única respuesta que el sistema principal va a comprender.

  Por ejemplo cuando una transacción es aprobada en PSE, este responde con 'resultCode: "00" ', en Nequi con 'success: True', el adaptador traduce todo esto a un estado único interno: 'APROBADO'. En el caso de Paypal no es necesario hacer una transformación en este parámetro ya que Paypal responde con el mismo formato 'estado: "APROBADO", pero si es necesario el adaptador en este método para el parámetro de "autorización".

- Aísla por completo el sistema principal del proveedor: Al implementar el adaptador, el sistema consigue un aislamiento porque el sistema de órdenes (sistema_ordenes.py) ya no importa directamente las clases de cada uno de los proveedores sino que solo conoce la interfaz ServicioPago, la cual es implementada en cada uno de los adaptadores. Esto permite que la complejidad técnica de conexión externa sea únicamente responsabilidad del adaptador.

```python
class AdaptadorPSE(ServicioPago):
    """Adapta PSE al contrato interno."""

    def __init__(self):
        self._proveedor = PSE()

    def procesarPago(self, clienteId: str, monto: float) -> dict:
        # Traduce parámetros internos → formato PSE
        r = self._proveedor.executeTransaction(
            user=clienteId,
            amount=monto,
            currency="COP"
        )
        # Traduce respuesta PSE → contrato interno
        return {
            "estado":              "APROBADO" if r["resultCode"] == "00" else "RECHAZADO",
            "codigoAutorizacion":  r["authId"]
        }
```

*Figura 11. Adaptador para PSE*

**Paso 5 –** Validación mediante prueba automatizada mínima

La integración de pruebas mediante unittest permitió evaluar la estabilidad del sistema. Se implementaron dos tipos de pruebas:

- Validacion de las transformaciones (TestAdaptadores): Esta permitieron visualizar que los adaptadores cumplieran correctamente el trabajo de “transformar”. Garantizar que el lenguaje técnico del proveedor como el campo “autorizacion” en el caso de Paypal, no se filtre en el sistema. Además, asegura que independientemente del proveedor el resultado siempre sea el valor esperado por el sistema interno “estado: APROBADO” para cuando el proceso de pago es satisfactorio.
- Validación de intercambiabilidad (TestIntercambiabilidad): Demuestra que el sistema principal (SistemaOrdenes) funciona correctamente procesando pagos independientemente del adaptador que se esté usando. El método sistema.crearOrden funciona con cualquiera de los proveedores sin tener que modificar la lógica del sistema de órdenes.

```python
class TestAdaptadores(unittest.TestCase):
    """Pruebas básicas para asegurar que los adaptadores funcionen"""

    def test_pse_aprobado(self):
        # El sistema debe recibir 'APROBADO' desde el adaptador PSE
        r = AdaptadorPSE().procesarPago("usuario-1", 5000)
        self.assertEqual(r["estado"], "APROBADO")

    def test_paypal_aprobado(self):
        # El sistema debe recibir 'APROBADO' desde el adaptador Paypal
        r = AdaptadorPaypal().procesarPago("usuario-2", 5000)
        self.assertEqual(r["estado"], "APROBADO")
        # Paypal devuelve 'autorizacion',pero el sistema espera 'codigoAutorizacion'. Verificamos que se traduzca.
        self.assertNotIn("autorizacion", r)

    def test_nequi_aprobado(self):
        # El sistema debe recibir 'APROBADO' desde el adaptador Nequi
        r = AdaptadorNequi().procesarPago("usuario-3", 5000)
        self.assertEqual(r["estado"], "APROBADO")

class TestIntercambiabilidad(unittest.TestCase):
    """El sistema de órdenes funciona con cualquier proveedor"""

    def test_sistema_con_pse(self):
        sistema = SistemaOrdenes(AdaptadorPSE())
        orden = sistema.crearOrden("cliente-1", 1000)
        self.assertEqual(orden["estado"], "APROBADO")

    def test_sistema_con_paypal(self):
        sistema = SistemaOrdenes(AdaptadorPaypal())
        orden = sistema.crearOrden("cliente-2", 1000)
        self.assertEqual(orden["estado"], "APROBADO")

    def test_sistema_con_nequi(self):
        sistema = SistemaOrdenes(AdaptadorNequi())
        orden = sistema.crearOrden("cliente-3", 1000)
        self.assertEqual(orden["estado"], "APROBADO")
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   powershell

PS C:\Users\ROBINSON\Documents\UPTC\Software II\Taller 2\Taller_2_Interoperabilidad> python -m unittest discover tests -v

==================================================
   EJECUTANDO PRUEBAS DE INTEROPERABILIDAD
==================================================

test_nequi_aprobado (test_adaptador.TestAdaptadores.test_nequi_aprobado) ...     → [Nequi] submit_payment(account_ref='usuario-3', value=5000, iso_currency='COP')
ok
test_paypal_aprobado (test_adaptador.TestAdaptadores.test_paypal_aprobado) ...   [PayPal] executeTransaction(user='usuario-2', amount=5,000, currency='COP, descripcion='Pago Orden', producto='1')
ok
test_pse_aprobado (test_adaptador.TestAdaptadores.test_pse_aprobado) ...     → [PSE] executeTransaction(user='usuario-1', amount=5000, currency='COP')
ok
test_sistema_con_nequi (test_adaptador.TestIntercambiabilidad.test_sistema_con_nequi) ...     → [Nequi] submit_payment(account_ref='cliente-3', value=1000, iso_currency='COP')
ok
test_sistema_con_paypal (test_adaptador.TestIntercambiabilidad.test_sistema_con_paypal) ...   [PayPal] executeTransaction(user='cliente-2', amount=1,000, currency='COP, descripcion='Pago Orden', pro
ducto='1')
ok
test_sistema_con_pse (test_adaptador.TestIntercambiabilidad.test_sistema_con_pse) ...     → [PSE] executeTransaction(user='cliente-1', amount=1000, currency='COP')
ok

----------------------------------------------------------------------
Ran 6 tests in 0.066s
```

*Figura 12. Pruebas automatizadas con unittest*

**Paso 6 –** Simulación de segundo proveedor (opcional para nota sobresaliente)

Se implementó un segundo proveedor (Nequi) el cual tiene nombres de campos distintos en comparación con el proveedor de PSE o Paypal.

```python
class AdaptadorNequi(ServicioPago):
    """
    Adapta Nequi (interfaz totalmente diferente) al contrato interno.
    El sistema principal lo usa exactamente igual que los adaptadores PSE y Paypal.
    """

    def __init__(self):
        self._proveedor = Nequi()

    def procesarPago(self, clienteId: str, monto: float) -> dict:
        r = self._proveedor.submit_payment(
            account_ref=clienteId,
            value=monto,
            iso_currency="COP"
        )
        return {
            "estado":             "APROBADO" if r["success"] else "RECHAZADO",
            "codigoAutorizacion": r["confirmation_code"]
        }
```

*Figura 13. Adaptador para Paypal.*

El Sistema de ordenes (SistemaOrdenes.py) puede procesar pagos de Nequi usando el mismo método que usa para los pagos de Paypal y PSE. Esto permite visualizar que el sistema puede agregar 10 formas de pago más sin modificar el núcleo principal de la aplicación sino que solamente se tendría que crear el adaptador que transforma los campos tanto del formato del proveedor al interno del sistema, como del formato interno al externo de cada uno de los proveedores.

```python
elif opc == "3":
    print("\n  a) PSE\n  b) Paypal\n  c) Nequi")
    sel = input("  Elige proveedor: ").strip().lower()

    adps = {"a": AdaptadorPSE(), "b": AdaptadorPaypal(), "c": AdaptadorNequi()}
    adaptador = adps.get(sel, AdaptadorPSE())

    cliente, monto = pedir_datos()
    orden = SistemaOrdenes(adaptador).crearOrden(cliente, monto)
    mostrar_orden(orden)
```

*Figura 14.Diferentes instancias para un mismo sistema.*

A continuación se ve el resultado, de cómo el sistema interno únicamente requiere del el “Cliente ID” y el “Monto”, he independientemente del método de pago que se use el sistema es capaz de procesarlo, solo depende del contrato estable:

```
Selecciona una opción: 3

a) PSE
b) Paypal
c) Nequi
Elige proveedor: c

Cliente ID (Enter = 'cli-001'): cli-001
Monto COP  (Enter = 50000)  : 10000
  → [Nequi] submit_payment(account_ref='cli-001', value=10000.0, iso_currency='COP')

--- Resultado de la Transacción ---
Estado              : APROBADO
Orden ID            : ORD-cli-001-001
Código Autorización : NQI-7731-OK
```

*Figura 15.Resultado proceso de pago con Nequi.*

```
Selecciona una opción: 3

a) PSE
b) Paypal
c) Nequi
Elige proveedor: b

Cliente ID (Enter = 'cli-001'): cli-008
Monto COP  (Enter = 50000)  : 7000
[PayPal] executeTransaction(user='cli-008', amount=7,000, currency='COP, descripcion='Pago Orden', producto='1')

--- Resultado de la Transacción ---
Estado              : APROBADO
Orden ID            : ORD-cli-008-001
Código Autorización : PAYPAL-AUTH-4567
```

*Figura 16.Resultado proceso de pago con Paypal.*

1. ¿Qué tipo de acoplamiento existía inicialmente?

   ***Respuesta***: Inicialmente existía un acoplamiento mayor, un acoplamiento concreto directo. Pues el sistema de órdenes acoplado (acoplado.py) importa y crea una instancia directa con el proveedor principal PSE, por lo que el sistema principal conoce exactamente que clase va a utilizar, además se llama al método executeTransaction con los nombres de parámetros específicos de la clase PSE, por lo cual si PSE llega a renombrar se rompe la conexión, y por último, el sistema principal conoce lo que retorna el método.

2. ¿Qué cambios ahora quedan encapsulados?

   ***Respuesta***: Con el uso de los adaptadores destinados para cada proveedor, el sistema principal de órdenes solo necesita conocer la función para procesar pagos expuesta en la interfaz. De esta manera, si se presenta un cambio en algún proveedor ya sea por aumento de parámetros o modificación de nombre de los mismos, este queda encapsulado dentro de su respectivo adaptador y no afecta el resultado ni el resto del sistema, como sí ocurría anteriormente.

3. ¿El adaptador mejora la modificabilidad? Explique.

   ***Respuesta***: Si, en definitiva el adaptador mejora la modificabilidad. Principalmente porque si no hubiera o existiera un adaptador cualquier cambio o modificación en el procesos obligaría a modificar el sistema principal de órdenes, lo que generaría inconvenientes, más trabajo, más errores, más tiempo y más recursos. Por el contrario, con el uso del adaptador se manejan mejor estos cambios solo dentro de esta capa. Este adaptador se encarga de actuar como traductor entre el sistema y el proveedor externo manteniendo todo el sistema estable.

4. ¿Qué costo introduce esta solución?

   ***Respuesta***: Está solución añade más capas al sistema, debido a que ahora se tienen que implementar interfaces y clases adaptadoras el programador tendrá un flujo de seguimiento mucho más grande por el aumento de archivos. Así mismo, la potencia aumenta debido a que los datos ahora tienen que pasar por el adaptador para ser transformados antes de llegar al sistema principal, lo que disminuye el rendimiento del sistema y esto mismo implica tener una arquitectura mucho más robusta contrario a la de realizar una conexión directa o ingenua, lo que conlleva a que se requiera más tiempo.

5. ¿En qué escenario el adaptador no sería suficiente?

   ***Respuesta***: En sistemas en los que el proveedor no usa un modelo sincrónico en sus pagos sino que lo hace de manera asincrónica en el cual después de realizar el pago y 3 horas después recibes la confirmación, el adaptador no podría convertir esa espera de 3 horas en una respuesta inmediata. En sistemas de alta frecuencia como el trading donde se busca que la latencia sea lo más baja posible, el uso de un adaptador no sería lo ideal, ya que al realizar una operación los datos tendrían que pasar por más capas y podría llevar a pérdidas económicas de la persona por no realizar la compra o venta en el momento exacto donde dio click.

   Cuando un proveedor requiere información adicional y el sistema principal de órdenes no posee esa información el adaptador no puede inventar esa información para enviarla.